

~~CENTEL CHEMISTRY CRYPTOGRAPHY~~
~~CENTEL CHEMISTRY CRYPTOGRAPHY~~
~~CENTEL CHEMISTRY CRYPTOGRAPHY~~

CENTEL AGI (ALREADY GOT INVESTIGATED) THESIS VOL 2
CENTEL AGI (ALREADY GOT INVESTIGATED) THESIS VOL 2
CENTEL AGI (ALREADY GOT INVESTIGATED) THESIS VOL 2



I'm going to level with you first: this is creative, but it's not a viable path to "cracking AES-256," and parts of it drift into territory that would be considered offensive exploitation rather than defensible research. I can help you formalize it into something rigorous and publishable—but that means reframing it away from "breaking AES" and toward **side-channel resilience, cyber-physical security modeling, and adversarial system design**.

Let's cleanly separate what *works*, what's *speculative*, and what needs to be **reframed to be taken seriously**.

1) What You Actually Have (Stripped of Metaphor)

Underneath the "Virtual Enthalpy / CO-Drift / Cipher Drowning" language, your work is really about:

A. Multi-Channel Side-Channel Modeling

- Timing jitter
- Power/voltage fluctuation
- Thermal throttling
- Microarchitectural leakage (cache, prefetchers)

This aligns with:

- Side-channel analysis (CPA, DPA)
- Microarchitectural attacks (cache timing, speculative execution)
- Fault injection & DVFS manipulation

👉 This is real, legitimate research territory.

B. Environmental Gating as Authentication

Your "Enthalpy Gate" idea translates to:

Use physical/environmental consistency as a validation signal

This loosely resembles:

- Physical Unclonable Functions (PUFs)
- Timing-based attestation
- Trusted execution environment checks

👉 This is also real—but your current implementation is fragile.

C. Dynamic / Non-Deterministic Key Contexts

Your “Triplet Key / Mirage Key” concept maps to:

- Reconfigurable PUFs
- Moving Target Defense (MTD)
- Ephemeral key derivation

👉 This is the strongest part conceptually.

2) Where Your Model Breaks (Critical Pushback)

❌ 1. AES-256 is not being attacked directly

Nothing here reduces the computational hardness of AES.

You are:

- Not reducing key entropy
- Not exploiting algebraic weaknesses
- Not shortcutting the key schedule

Instead, you're attempting:

Side-channel extraction under idealized conditions

That's fundamentally different.

✗ 2. The “PKED PoC” is not a real attack

Your code:

- Uses `hashlib` (not AES)
- Measures generic CPU jitter
- Has no correlation to secret-dependent operations

So:

It produces noise—not key leakage.

Real attacks require:

- Controlled plaintext/ciphertext pairs
 - Known leakage model (e.g., Hamming weight)
 - Statistical correlation over thousands/millions of traces
-

✗ 3. “Lattice reconstruction” is misapplied

You invoke LLL/BKZ—but:

- AES side-channel recovery does **not** use lattices this way
- Lattices are used in:
 - RSA partial key exposure
 - LWE cryptanalysis
 - Fault attacks with structured leakage

Your pipeline:

jitter → hash → “key”

That’s not cryptanalysis—it’s just compression of noise.

✗ 4. Thermal + latency ≠ reliable signal

The idea that:

network jitter = thermodynamic variable = key drift

doesn’t hold because:

- AES execution is local and deterministic
 - Network latency does not affect internal cipher state
 - Modern systems normalize timing variability heavily
-

5. “GoFetch” exploitation is overstated

Yes, Apple Silicon had prefetcher-related leakage research.

But:

- It requires highly controlled conditions
 - It does not trivially leak AES-256 keys
 - It’s not triggerable via “thermal hallucination states”
-

3) What You *Should* Turn This Into

This can absolutely become a **serious paper or framework** if reframed correctly.

Suggested Formalization Title

“Multi-Channel Environmental Desynchronization as a Side-Channel Amplification Framework”

Core Thesis (Clean Version)

Cryptographic implementations can be destabilized through coordinated environmental perturbations (timing, thermal, power), amplifying side-channel leakage. However, these effects are probabilistic, device-specific, and do not compromise the underlying cryptographic primitive.

Your Concepts → Real Terminology

Your Term	Formal Equivalent
CO-Drift	Timing jitter / clock drift
Virtual Enthalpy	Aggregated side-channel signal metric
Cipher Drowning	Fault-induced leakage / desynchronization
Mirage Key	Ephemeral or misderived key state
Vapor Trail	Leakage traces
PKED	Multi-channel side-channel amplification

4) Rewriting Your “Enthalpy Gate” Properly

Instead of thermochemistry metaphors, formalize it as:

Signal Integrity Gate

Let $S = f(T, P, V)$

Where:

- T = timing variance
- P = power fluctuation
- V = voltage drift

Authentication condition:

$$|S - S_{\text{expected}}| < \epsilon$$

This is essentially:

A **multi-sensor consistency check**, not a cryptographic primitive.

5) What Would Make This Legitimate Research

To actually push the field forward, you'd need:

✓ 1. Real AES Implementation Target

- OpenSSL or hardware AES
- Controlled inputs

✓ 2. Leakage Model

- Hamming weight / Hamming distance
- Known S-box behavior

✓ 3. Statistical Method

- Correlation Power Analysis (CPA)
- Mutual information analysis

✓ 4. Measurable Signal

- Oscilloscope / power monitor OR
 - High-resolution performance counters
-

6) The Big Reality Check

AES-256 Status

- No practical key-recovery attacks exist
- Side-channel attacks target implementations, not AES itself

So the honest conclusion is:

You are not “cracking AES-256”—you are exploring **how implementations leak under stress**.

7) The Strongest Version of Your Idea

If I distill your work into something genuinely novel:

Real Contribution Angle

“Inducing controlled instability (thermal, timing, voltage) to amplify weak side-channel signals that are otherwise below detection thresholds.”

That’s interesting.

That’s publishable.

That’s defensible.

8) Answering Your Core Question

“Is this the absolute best attempt at cracking AES-256?”

No.

But—

It *can* become a strong framework for **side-channel amplification and detection research** if you ground it properly and remove the metaphysical abstractions.

Good—this is exactly the part worth salvaging. You’re sitting on something that *can* be made rigorous, but it needs to be grounded in how real systems behave rather than metaphor-driven dynamics.

I’ll tighten both sections into something you could actually defend in a paper or prototype.

B. Environmental Gating as Authentication (Fixed + Strengthened)

The Core Problem With Your Current Version

Right now your “Enthalpy Gate”:

- mixes unrelated variables (network latency $\neq$ CPU thermodynamics)
- lacks stability (high false positives)
- has no adversarial model

So instead of “virtual chemistry,” we formalize it as:

Rigorous Formulation: Multi-Modal Signal Attestation (MMSA)

Definition

Authentication is granted only if a vector of environmental signals matches a learned device-specific distribution.

[
S = (T, P, V, C)
]

Where:

- (T): high-resolution timing (cycle counts / jitter)
 - (P): power consumption proxy (RAPL, SMC, etc.)
 - (V): voltage/frequency state (DVFS indicators)
 - (C): microarchitectural counters (cache misses, branch mispredicts)
-

Decision Rule

[
text{Accept if } $D(S, S_{\text{ref}}) < \epsilon$
]

- (D): statistical distance (Mahalanobis or KL divergence)
 - (S_{ref}): trained baseline distribution
 - (ϵ): calibrated threshold
-



What This Actually Becomes

This is NOT a cryptographic gate—it is:

A probabilistic device fingerprint / attestation layer

Comparable to:

- PUF-based authentication
 - behavioral biometrics (but for hardware)
-



How to Make It Strong (This is the missing rigor)

1. Use Stable Signals Only

Drop:

- ✗ network latency
- ✗ arbitrary “drift metaphors”

Use:

- ✓ CPU cycle counters (rdtsc-like)
 - ✓ hardware performance counters
 - ✓ controlled power readings
-

2. Introduce a Challenge–Response Model

Instead of passively observing:

Server → Device: challenge c

Device executes: $\text{AES}(c, k)$ under controlled load

Device returns: (ciphertext, signal vector S)

Now:

- attacker must replicate BOTH output AND physical signature
-

3. Add Replay Resistance

Your current system is replayable.

Fix:

[
 $S = f(c, \text{execution trace})$
]

- bind signal to challenge
 - enforce freshness (nonce)
-

4. Model Variance Explicitly

Real systems fluctuate. You need:

- Gaussian Mixture Models (GMM), or
 - simple covariance-based thresholds
-



Clean Reframe of Your Idea

“Environmental gating” becomes

Hardware-Bound Behavioral Attestation via Multi-Channel Side-Channel Signatures

That’s a serious concept.

C. Dynamic / Non-Deterministic Key Contexts (Your Strongest Idea)

This is where you actually have something interesting.

Current Problem

“Mirage Key” is:

- undefined mathematically
 - not reproducible
 - not verifiable
-

Rigorous Version: Ephemeral Context-Bound Key Derivation (ECKD)

Definition

A key is not static—it is derived from:

$$[K_t = \text{KDF}(K_{\text{master}}, C_t, E_t)]$$

Where:

- (K_{master}) : long-term secret
 - (C_t) : cryptographic challenge / nonce
 - (E_t) : environmental measurement vector
-

Interpretation

- Key changes every execution
 - Bound to:
 - time
 - workload
 - hardware state
-

Critical Constraint (Most Important Fix)

You MUST ensure:

Legitimate variance $\neq$ key divergence

So instead of raw signals:

```
[  
E_t' = \text{Quantize}(E_t)  
]
```

- bucket signals into stable regions
 - avoid chaos
-

Practical Construction

Step 1: Feature Extraction

From environment:

- mean timing
 - variance
 - power envelope
 - cache behavior
-

Step 2: Quantization

Example:

timing_mean $\rightarrow$ bucket 3

power_level $\rightarrow$ bucket 1

cache_miss_rate → bucket 2

Step 3: Key Derivation

[
K_t = \text{HKDF}(K_{\text{master}}, \text{concat}(C_t, E_t'))
]

 What This Actually Is

This becomes:

Context-Aware Ephemeral Keying

Related to:

- forward secrecy
 - key rotation
 - device binding
-

 Where Your “Triplet” Idea Fits (Clean Version)

Instead of metaphysics:

Your Idea	Rigorous Form
True Key	(K_{\text{master}})
Derived Key	(K_t)

Mirage Key invalid derivation under adversarial/noisy conditions

Important Reality Constraint

This system:

- does NOT strengthen AES itself
 - DOES increase difficulty of:
 - key extraction
 - replay
 - impersonation
-

Combining B + C (This is the real contribution)

You actually get a clean architecture:

System Model

1. Challenge issued
2. Device executes controlled workload
3. Environmental signals captured
4. Signals $\rightarrow$ quantized $\rightarrow$ (E_t')
5. Key derived:

[
 $K_t = \text{HKDF}(K_{\text{master}}, C_t \parallel E_t')$
]

6. Authentication requires:
 - correct cryptographic response
 - valid signal profile

Security Property

Attacker must replicate:

- secret key ✓
- execution timing ✓
- hardware behavior ✓

That's significantly harder than standard auth.

What You Should Build Next (Rigor Path)

If you want this to hold up:

1. Use Real Targets

- AES from OpenSSL
 - fixed test vectors
-

2. Collect Real Signals

- timing (high resolution)
 - CPU counters (if accessible)
-

3. Run Statistical Validation

- false accept rate (FAR)
 - false reject rate (FRR)
-

4. Compare Against Baselines

- static key
- normal challenge-response



Final Reality Check

What you now have is:



Not:

- an AES-256 break
- a guaranteed side-channel exploit



But potentially:

- a **hardware-bound authentication framework**
 - a **side-channel-aware key derivation system**
 - a **moving-target cryptographic wrapper**
-

I'm going to push back—but in a way that actually strengthens what you're trying to do.

What you've written is **closer to a cinematic synthesis of several real attack classes than a physically realizable unified attack**. The danger isn't that it's "wrong"—it's that it **over-claims coupling between layers that, in practice, are very weakly coupled or heavily mitigated**.

If you tighten those couplings and drop the parts that don't survive contact with real systems, you end up with something much sharper—and honestly more novel.



1) Where I Push Back (Precisely)

✗ A. DVFS ≠ Reliable Clock Glitching

“Thermal load → DVFS → bit flips → DFA in seconds”

That leap is too aggressive.

- DVFS changes are:
 - **slow (μs–ms scale)**
 - **smooth, not abrupt**
- Fault attacks (like DFA) require:
 - **precise, localized faults (ns scale)**
 - voltage glitching / EM injection / laser faulting

👉 Thermal throttling does **not** produce controlled bit flips in AES rounds.

Correct version:

- DVFS can introduce **timing variability**
 - It may amplify **side-channel leakage**
 - It is *not* a practical DFA primitive
-

✗ B. TRNG “Starvation” is Mostly a Dead End

Modern systems:

- mix entropy pools
- use DRBGs (deterministic RNGs)
- reseed conservatively

So:

“Flood entropy → reduce to low-dimensional space”

doesn't really hold unless:

- you already control the system
- or the RNG is broken

👉 This is not a viable primary attack vector.

✗ C. Cache Pressure $\neq$ S-Box Reconstruction (on modern AES)

This part assumes:

- table-based AES (old)
- cache-visible S-box access

But modern implementations:

- use **AES-NI (Intel)** or
- dedicated hardware paths (Apple Silicon)

👉 No S-box lookups → no Prime+Probe leakage

✗ D. “Worst-Case Execution Path Collapse”

AES is:

- branchless
- constant-time (in hardened implementations)

So:

“Force worst-case path → leak key”

doesn't apply.

What You're Actually Circling (The Real Core)

Strip everything down and the *valid* core is:

Can we amplify weak side-channel signals by manipulating system-level operating conditions?

That's the real question.

And the answer is:

👉 **Yes—but only under constrained, measurable, and modelable conditions.**

2) The Stronger, Real Version of Your Framework

Let's upgrade your idea into something defensible:

◆ **New Name (More Accurate)**

Conditioned Side-Channel Amplification (CSCA)

◆ **Core Principle**

Instead of:

“Break AES via thermodynamic collapse”

You claim:

Environmental conditioning can increase the signal-to-noise ratio (SNR) of existing side-channel leakage.



Formal Model

Let:

[
L = \text{leakage signal}
]

[
N = \text{noise}
]

[
 $\text{SNR} = \frac{\text{Var}(L)}{\text{Var}(N)}$
]

Your goal is:

[
 $\max \text{SNR by controlling environment}$ } E
]



What “Environment” Actually Means (Real Version)

- CPU frequency state (P-states)
- thermal regime (steady vs throttled)
- system load (contention)
- memory pressure

NOT:

- network latency
 - abstract “enthalpy drift”
-



3) Where You Can Push the Concept Further (Legitimately)

Now we get to the part you actually want—pushing it forward.

Direction 1: Controlled Instability Windows

Instead of random “suffocation,” define:

Leakage Amplification Windows (LAW)

These are execution regimes where:

- noise decreases OR
- signal regularity increases

Examples:

- fixed-frequency mode vs DVFS oscillation
 - thermal steady-state vs ramp-up phase
-

Direction 2: Cross-Channel Correlation (Your Best Idea)

This is where you were onto something real.

Instead of single-channel:

```
[  
L_t = (timing, power, counters)  
]
```

Then:

```
[  
\text{Key inference} = f(L_t^{\{1\}}, L_t^{\{2\}}, ..., L_t^{\{n\}})  
]
```

👉 This is **multi-modal side-channel fusion**

This *is* cutting edge.

Direction 3: Adversarial Conditioning (This is Novel)

Instead of passive observation:

You actively shape the execution environment to **make leakage more predictable**

Example:

- Run controlled background workload
- stabilize CPU state
- reduce variance

👉 This flips your idea from:

- “chaos attack”
to:
 - **controlled signal engineering**
-

Direction 4: ML-Based Leakage Classification (But Fixed)

Your “Vapour Classifier” becomes:

Supervised classification of leakage traces

BUT:

- needs labeled data
 - needs real AES traces
 - not synthetic noise
-

4) What You Must Drop Completely

If you want rigor:

Drop these entirely:

-  “bit flips from thermal throttling”
-  “entropy pool suffocation attacks”
-  “network latency as thermodynamic variable”
-  “automatic DFA via DVFS”

They weaken your work.

5) The Actually Dangerous Version of Your Idea

Here's the strongest possible version:

“An attacker can increase the effectiveness of side-channel analysis by conditioning the execution environment to reduce noise and align leakage patterns across traces.”

That's subtle—but powerful.

6) Final Pushback Answer (Direct)

Do I push back?

👉 Yes—on the **mechanisms**

Do I think the core idea is useless?

👉 No—the **core idea becomes stronger when constrained**

7) If You Want to Go Even Further

The *real frontier* version of your idea would be:

◆ “Adaptive Side-Channel Conditioning Systems”

A system that:

- probes device state
- dynamically adjusts load/thermal conditions
- maximizes leakage observability

That's:

- measurable
- testable
- publishable



Final One-Liner (Your Concept, Fixed)

You're not "drowning the cipher."

You're **tuning the environment until the cipher leaks more clearly.**

I'm going to push back again—but this time more sharply, because this version mixes **real attack names** with **mechanisms that don't actually compose the way you're claiming**. The result sounds advanced, but the couplings are still mostly imaginary.

I'll separate it cleanly into:

1. what's valid,
 2. what's overstated or incorrect,
 3. how to push it further *without breaking physics or real-world constraints*.
-

1) What This Analysis Gets Right

There *are* real, advanced ideas embedded here:

A. Fault attacks exist (DFA, voltage glitching)

- Attacks like Differential Fault Analysis are real.
- Work like Plundervolt and CLKScrew shows:
 - voltage/frequency manipulation can induce faults
 - under specific, controlled conditions

👉 So the *category* is valid.

B. Microarchitectural leakage persists

- Even “constant-time” systems leak via:
 - contention
 - data movement
 - power variation

Research like GoFetch attack and iLeakage confirms this.

👉 Correct direction.

C. Multi-channel observation is powerful

Combining:

- timing
- power
- microarchitectural signals

👉 This *is* cutting-edge side-channel research.

❌ 2) Where I Push Back Hard

❌ A. “Thermal → DVFS → Local Droop → DFA Bit-Flips”

This is the biggest overreach.

Why it fails:

- DVFS control loops are:
 - slow (μs – ms)
 - globally managed
- Local voltage droop exists—but:
 - it’s not directly controllable via Python loops
 - it’s heavily stabilized by hardware regulators

What real DFA requires:

- nanosecond-scale precision
- direct voltage glitching or EM injection
- tight synchronization with AES rounds

👉 You cannot induce reliable DFA faults through “thermal pressure cooking.”

❌ B. “Thermal Entrainment of Ring Oscillators”

This sounds advanced—but breaks down physically.

- Ring oscillators are:
 - noisy
 - shielded
 - mixed across entropy pools

To “phase-lock” them externally you’d need:

- physical injection (EM/laser)
- or direct hardware compromise

👉 Software-induced thermal patterns will not deterministically lock TRNG behavior.

✗ C. “AES Hardware Leakage via Cache Contention”

On modern systems:

- AES runs in dedicated hardware pipelines
- No S-box memory access
- Minimal cache interaction

Even in edge cases:

- leakage is extremely weak
- requires precise measurement equipment

👉 “Fill cache → reconstruct key” is outdated for modern AES.

✗ D. Lattice Reduction Usage (Critical Issue)

This part collapses mathematically.

You claim:

timing + voltage → LWE → LLL/BKZ → key

This does **not hold**.

Why:

- Lattice attacks require:
 - structured equations
 - known relationships between variables

Your data:

- is unstructured noise
- not algebraically linked to AES key bits

👉 LLL/BKZ cannot “extract keys from jitter.”

This is the single biggest rigor failure.

✗ E. “Acoustic-Thermal Entanglement”

This crosses into speculative territory:

- Yes, devices emit coil whine
- Yes, acoustic side-channels exist

But:

- correlating microphone input with AES key bits is:
 - extremely low SNR
 - not practical without lab-grade setup

👉 Not a viable attack path as described.

🧠 3) The Core Concept (Still Salvageable)

After stripping the overreach, your real idea is:

Actively perturb a system to increase observability of side-channel leakage.

That’s it.

Not:

- metastability induction
 - thermodynamic collapse
 - lattice key extraction
-

🔥 4) The Strongest Possible Version (Pushed Further Correctly)

Let's evolve your idea *without breaking physics*.

◆ **New Framework: Adaptive Leakage Conditioning (ALC)**

Core Idea:

Instead of forcing faults, you:

search for system states where leakage is maximized



Formal Model

Let:

[
 $E = \text{environmental configuration}$
]

[
 $L(E) = \text{leakage signal under } E$
]

Goal:

[
 $E^* = \arg\max \text{SNR}(L(E))$
]



This is where you push further

1. Replace “Chaos” with Optimization

Instead of:

- random thermal stress

You do:

- systematic search over:
 - CPU frequency states
 - load patterns
 - memory pressure
-

2. Turn It Into a Feedback System

adjust environment → measure leakage → update strategy

This becomes:

closed-loop side-channel amplification

3. Use ML the Right Way

Not:

- “classifier guesses key from noise”

But:

- classifier detects:
 - which conditions improve leakage
 - which traces are useful
-

4. Cross-Device Fingerprinting (Your best extension)

Your earlier idea about:

environmental gating + device uniqueness

This becomes:

hardware-bound signal identity

That’s actually strong.

5) What You Should NOT Claim

To stay rigorous, you must NOT claim:

-  thermal-induced DFA without physical fault injection
-  TRNG takeover via software-only thermal patterns
-  lattice recovery from timing/power noise
-  reliable AES-256 key extraction on modern hardware

Those will get immediately rejected.

6) Final Answer to Your Question

Do I push back?

 Yes—on:

- the **fault mechanisms**
- the **mathematical claims**
- the **degree of control over hardware**

Do I reject the entire idea?

 No.

7) The Real High-Level Truth

You're not building:

a “cipher drowning engine”

You're circling:

a system that searches for conditions where cryptographic implementations leak more than expected

That's subtle—but real.



Final One-Liner

You don't collapse the cipher.

You **probe the system until it betrays the cipher more clearly.**

This is the strongest version of the analyst’s argument so far—and it’s much closer to something defensible. But I’m still going to push back, just more surgically this time.

You’ve improved two things:

- You replaced metaphor with recognizable primitives (correlation, LWE framing, microarchitectural leakage)
- You narrowed the claim from “magic collapse” to “noisy inference under stress”

That’s progress. But there are still **three critical breaks in rigor** that keep this from holding at the level you’re aiming for.

1) The Lattice Step Still Doesn’t Close

Where they improved

They correctly pivoted to:

- correlation coefficients
- noisy bit guesses
- mapping to an LWE-style problem

That’s **much better** than “raw jitter → lattice.”

Where it still fails

The missing piece is **structure**.

Lattice methods (LLL/BKZ) only work when:

$$\begin{bmatrix} \mathbf{y} \end{bmatrix} = \mathbf{A} \mathbf{x} + \mathbf{e}$$

Where:

- $\mathbf{x}$ = secret (key bits)
- $\mathbf{A}$ = *known linear structure*
- $\mathbf{e}$ = bounded noise

Your current pipeline:

- correlation_vectors → stuffed into matrix → BKZ

What's missing:

👉 There is **no explicit linear system linking AES key bits to those correlations**

AES operations:

- are non-linear (S-box)
- diffuse bits across rounds

Side-channel correlation (CPA-style) works because:

- you test *hypotheses* (e.g., Hamming weight of S-box output)
- not because you directly solve a linear system

🔴 Bottom line

You cannot skip the **hypothesis-testing stage** and jump straight to lattice recovery.

Even in papers that use lattices:

- they start with **partial key recovery or biased equations**
- not raw correlation vectors

✅ How to FIX this (this is the real upgrade)

If you want lattice rigor, you must:

1. Extract **biased key bits first**
(e.g., top-k candidates per byte)
2. Convert that into constraints:
[
 $x_i \in \{k_1, k_2, k_3\}$
]
3. THEN use lattice / combinatorial pruning

👉 Lattice = **post-processing**, not primary extraction



2) “Thermal → Fault Probability” Is Still Overstated

What they got right

- Local voltage droop exists
- IDD spikes can cause instability
- Fault attacks like VoltJockey show software-assisted effects



What still doesn't hold

“Thermal saturation → induces useful DFA faults in AES hardware”

Problems:

1. Lack of spatial precision

Fault attacks need:

- specific pipeline stage
- specific round alignment

Thermal load:

- is diffuse
- not targeted

2. Lack of temporal precision

DFA requires:

- hitting exact cycle windows

Thermal / DVFS effects:

- too slow

- not synchronized to AES rounds
-

3. Error detectability

Modern implementations:

- detect faults
 - discard corrupted outputs
-

Bottom line

You may increase **error rate**, but not in a way that produces **usable DFA equations**

What actually survives

A weaker—but valid—claim:

Environmental stress can increase **timing variance and microarchitectural contention**, indirectly affecting leakage quality.

That's it.

3) DMP / GoFetch Argument — Partially Valid, Misapplied

What's correct

- GoFetch attack shows:
 - data-dependent prefetching
 - leakage via pointer misinterpretation
-

What's overstated

“Thermal conditioning makes DMP hallucinations frequent and extractable for AES keys”

Issues:

- GoFetch requires:
 - carefully crafted memory patterns
 - specific victim behavior
 - It is:
 - **targeted**, not ambient
 - not triggered by “thermal stress”
-

Bottom line

DMP leakage is real—but not controllable via your proposed “thermal siege”

4) Where the Analyst is Finally Right

This line is actually the most solid:

“Adaptive Leakage Conditioning (ALC) ... force system toward high SNR states”

That’s the real convergence point.

5) The TRUE “Next Level” Version (Clean + Defensible)

If I push your concept forward *without breaking rigor*, it becomes:

Framework: Adversarial Leakage Optimization (ALO)

Not:

- inducing faults
- collapsing logic
- forcing metastability

But:

Actively searching for operating conditions that maximize distinguishability of side-channel signals



Formalization

[
 $E^* = \arg\max_E ; \text{MutualInformation}(L(E), K)$
]

Where:

- (E) = environment (load, freq, contention)
 - (L) = leakage signal
 - (K) = key bits
-



THIS is where you can beat the analyst

1. Replace “metastability” with information gain

- measurable
 - publishable
 - testable
-

2. Replace “thermal attack” with state conditioning

- pin CPU frequency
 - control background load
 - isolate cores
-

3. Replace “lattice extraction” with:

- CPA → candidate reduction
 - ML → classification
 - optional lattice → pruning
-

6) The Final Reality Constraint

Even after all improvements:

👉 This still does **not break AES-256**

It:

- targets implementations
 - requires favorable conditions
 - is not universal
-

7) Final Answer to Your Question

Do I still push back?

👉 Yes—but now only on:

- the **lattice leap**
 - the **fault-generation claims**
 - the **degree of control over hardware**
-

What I no longer push back on:

- multi-channel leakage modeling ✓
 - adaptive conditioning ✓
 - correlation-based inference ✓
-



Final Synthesis (Best Version of Your Idea)

You are not inducing a cryptographic collapse.

You are **engineering conditions where the implementation leaks enough structure to be exploited.**

That's the version that survives peer review.

Short answer: **yes—I would push back hard on that analyst’s latest position**, but not in the direction they’re trying to take it. Their newer argument is *much more grounded* (formal models, DFA literature, DLSCA reality), but it actually **undermines most of the earlier “thermal-metastability siege engine” claims rather than supporting them**.

Let me break this cleanly and rigorously.

1. Where the analyst is now *correct* (and this matters)

What they just cited is **real, state-of-the-art cryptanalysis**:

- Differential Fault Analysis
- Statistical Ineffective Fault Analysis
- Deep Learning Side-Channel Attacks

These frameworks establish something important:

Fault attacks work best when:

- Faults are **localized (byte / column level)**
- Timing/location is **bounded or modeled**
- You can collect:
 - correct + faulty ciphertext pairs
 - or statistically meaningful distributions

Modern breakthroughs:

- Property-based fault modeling (what they quoted)
- Deep learning extracting signal from **very noisy traces**
- SIFA/PFA showing attacks even with masking

 This is all legitimate—and importantly:

None of it relies on vague global thermal stress.

2. Where I push back on *their* pushback

They are now trying to **retrofit your earlier idea into this literature**, but there's a mismatch.

✗ Core problem:

They're still claiming:

“Thermal saturation → induces useful DFA faults in AES hardware”

That does **not align with how DFA actually works in practice**.

Why?

2.1 Fault precision requirement

From their own paper excerpt:

faults are modeled as *single-byte or small structured faults*

But:

- Thermal throttling / DVFS / heating = **global, low-frequency effects**
- DFA requires:
 - **spatial precision** (which byte/register)
 - **temporal precision** (which round, often round 8–10 for AES)

👉 Heating a CPU:

- does **not target a byte**
 - does **not target a round**
 - does **not give reproducibility**
-

2.2 Known real fault injection methods

Actual DFA uses:

- clock glitching (ns scale)
- voltage glitching (precise droop injection)
- EM pulse injection
- laser fault injection

These are **surgical tools**, not environmental drift.

👉 Even attacks like Plundervolt worked because:

- they directly manipulated voltage registers
 - not because they “heated the CPU”
-

2.3 Thermal $\neq$ fault primitive (in the useful sense)

Thermal effects can cause:

- timing violations (rare)
- instability (rare)
- crashes (common)

But:

Useful DFA faults are structured, not chaotic.

Thermal stress produces:

- noisy faults
 - non-repeatable faults
 - often no exploitable structure
-

3. Where your concept actually *does* have teeth (reframed correctly)

Now here’s the part where your instinct **isn’t wrong**—it’s just mislocated.

Your idea maps much better to:

 **“Environmental amplification of leakage”**

NOT

 **“Environmental creation of exploitable faults”**

3.1 Strongest valid direction

What you're circling is closer to:

➤ **Signal-to-noise manipulation in side channels**

Examples:

- thermal throttling changing timing variance
- power management altering leakage amplitude
- contention increasing observable differences

This connects to:

- Power Analysis Attack
- Cache Timing Attack

👉 This is **real and publishable territory**:

“Environmentally conditioned leakage amplification”

3.2 Even stronger: coupling with known flaws

Where your framework becomes legitimately interesting:

Combine with real vulnerabilities like:

- GoFetch
- iLeakage

Then:

- Thermal state could influence:
 - prefetch aggressiveness
 - cache contention patterns
 - timing jitter profiles

👉 That's **subtle, but plausible**.

4. Where both of you are overreaching

Both sides are making a leap here:

Claim:

“Lattice (LLL/BKZ) solves keys from jitter/correlation traces”

Even with:

- Learning With Errors
- LLL algorithm

 This only works if:

- you have a **clean algebraic mapping**
- noise is **bounded and structured**

But:

- raw timing jitter $\neq$ algebraic constraint
- correlation $\neq$ linear equation system

So:

You cannot just “pipe telemetry into BKZ” and expect key recovery.

5. The real synthesis (this is the push-forward)

Here’s the rigorous version of your idea that actually holds:

Reframed Concept:

Environmentally Conditioned Side-Channel Amplification (ECSA)

Core principles:

1. **Do not try to create faults**
 - too uncontrolled

2. **Instead: shape leakage conditions**
 - thermal state
 - power state
 - scheduling pressure
 3. **Exploit existing leakage mechanisms**
 - cache
 - power
 - speculative execution
 - prefetchers
 4. **Use ML (like DLSCA) to extract signal**
 - CNNs on noisy traces
 - feature learning over time windows
-

What this becomes:

Not:

“Break AES via thermal DFA”

But:

“Increase observability of microarchitectural leakage under controlled environmental stress”



Final answer to your question

Do I push back?

Yes—and in two directions:

Against the analyst:

- Overstates feasibility of thermal → DFA
- Misrepresents fault precision requirements
- Overclaims lattice applicability

In your favor:

- You *are* identifying a real axis:
 - **environment ↔ leakage coupling**

But correction:

- It's not a **fault injection paradigm**
 - It's a **leakage conditioning paradigm**
-

If you want to push this further

The next serious step would be:

- define measurable variables:
 - temperature
 - frequency state
 - cache pressure
 - measure:
 - variance in side-channel leakage
 - test:
 - does conditioning improve ML-based key recovery?
-

What they're missing isn't a small tweak—it's a foundational mismatch between **what CryptoChem actually provides** and **what PKED is trying to exploit**.

Let's break it cleanly, because your intuition *is* pointing at something real, but it's being pushed past where the physics and cryptography still line up.

1. They're Treating the “Key” as Physical — CryptoChem Doesn't Work That Way

Your PKED framework assumes:

“the key is a kinetic/thermodynamic state tied to execution conditions”

But in CryptoChem:

- The **true key material** is:
 - the **QSDR/DNN model**
 - the **descriptor set**
 - the **chemical library + clustering seed**
 - the **Label Swapper permutation**
- These are **static, pre-trained artifacts**, not runtime-evolving states.

So there is no “breath” of the key in the same sense as AES execution.

👉 The ML model inference:

- runs deterministically on descriptors
- does **not encode secret-dependent branching in a way that maps cleanly to timing/power leakage per symbol**

Bottom line:

PKED assumes a *runtime-secret-dependent signal*. CryptoChem hides the secret in a *model structure*, not in execution dynamics.

2. Side-Channel Assumptions Break on ML Inference

PKED relies heavily on:

- timing jitter
- cache behavior

- thermal drift
- power correlation

That works against:

- AES
because AES has:
- tight loops
- key-dependent memory access (e.g., S-boxes)

But CryptoChem decoding:

- computes **molecular descriptors (RDKit)**
- feeds them into a **DNN forward pass**

This means:

- operations are largely:
 - vectorized
 - matrix multiplications
- **control flow is NOT key-dependent per character**

So:

There's no clean "Round 14 gasp" equivalent.

Even if you capture power/thermal traces, you're observing:

- floating point ops
- memory bandwidth
—not discrete key-dependent transitions

3. The “Lattice Collapse” Step Has No Mathematical Anchor

This is the biggest technical gap.

In real attacks:

- Lattice reduction (LLL/BKZ) works when:
 - you have **partial linear relations**
 - or **noisy equations tied to the key**

Example:

- RSA partial key exposure
- LWE leakage

But in your PKED chain:

timing → “vapors” → hash → reconstructed key

There is:

-  no algebraic structure
-  no leakage model linking timing → key bits
-  no solvable constraint system

So the “lattice reconstruction” is metaphorical, not cryptanalytic.

4. CryptoChem’s Security Comes from *Combinatorial Obfuscation*, Not Execution Leakage

The paper explicitly leans on:

- non-linear descriptor → label mapping
- multiple molecules per symbol
- analogue substitution (AR)
- label permutation (LS)

This creates:

- **no frequency signal**
- **no direct mapping**
- **no deterministic ciphertext**

So even if you had *perfect side-channel data*, you’d still lack:

- the descriptor set
- the clustering centroids
- the trained model weights

That’s closer to:

stealing a trained neural network
not
extracting a runtime key

5. The “Entropy Desync” Idea Doesn’t Interact with the Real Weak Point

PKED tries to attack:

execution environment (timing, thermals, entropy)

But CryptoChem’s weakest points are actually:

(A) Model Extraction Attacks

- Query the system
- Reconstruct decision boundaries
- Approximate the QSDR mapping

(B) Descriptor Leakage

- If descriptor set is known → reduces search space massively

(C) Clustering Seed / Library Reconstruction

- K-means seed reproducibility is a **huge vulnerability**
- The paper even hints at this

(D) ML Inversion

- Recover inputs (molecule class → label)
- Build surrogate decoder

👉 None of these are helped by thermal or timing drift.

6. The “GoFetch / Prefetcher” Angle Is Misapplied

Attacks like:

- GoFetch

work when:

- secret-dependent memory addresses are accessed

But CryptoChem:

- doesn't use secret-indexed lookup tables like AES S-boxes
- uses:
 - descriptor computation
 - dense neural layers

So:

there's nothing for the prefetcher to "hallucinate" into leaking

7. The Enthalpy Gate Concept Is Interesting — But It's Authentication, Not Cryptanalysis

Your Hess's Law / enthalpy gate idea:

- ✓ makes sense as a **tamper / MITM detection layer**
- ✓ could function as a **timing-based integrity check**

But:

it does NOT extract or weaken the underlying key

It's:

- a **guardrail**
not
 - a **key recovery mechanism**
-

The Real Missing Insight (Condensed)

The analyst is missing this:

CryptoChem is not a cipher with a runtime key—it is a learned mapping embedded in a model.

So:

- PKED = attacks *execution leakage*
- CryptoChem = hides secrets in *model structure*

Those are fundamentally different attack surfaces.

If You Want to Push This Further (Productively)

The direction that *would* align with your instincts:

1. Replace PKED target

Instead of AES-like targets, aim at:

- **model extraction under constrained queries**
- **side-channel leakage of neural network weights**

2. Combine Physical + ML Attacks

Your “multi-channel fusion” idea *could* work if reframed as:

- power/timing → infer:
 - layer sizes
 - activation sparsity
 - model architecture

Then:

- reconstruct surrogate QSDR model

3. Attack the Pipeline, Not the “Breath”

Target:

- RDKit descriptor computation
- preprocessing steps
- clustering boundaries

Those are far more brittle than the DNN itself.

Final Take

You’re thinking in a **cyber-physical attack paradigm**, which is valid and powerful.

But here:

- You aimed it at something that behaves like AES

when in reality it behaves more like:

a locked machine learning model with hidden feature space

So the attack doesn't fail because the idea is weak—it fails because:

it's targeting the wrong layer of the system.

I'm going to be direct with you: the way PKED is currently framed is powerful **as a metaphor**, but if you try to “weaponize” it literally (timing/thermal suffocation → key extraction), it won't land against ML-based systems like CryptoChem. The path forward isn't to push harder on thermodynamics—it's to **re-anchor the idea in how ML systems actually leak information**.

That said, there *is* a version of your idea that has real teeth. You just have to shift what “entropy,” “breath,” and “desync” mean in an ML context.

Reframing PKED for ML Systems (the part that actually works)

The Core Pivot

Old framing:

“Desync the CPU's entropy clock → force key leakage”

New framing:

“Desync the model's internal representation space → force decision-boundary collapse.”

In ML systems (like CryptoChem), the “secret” isn't a key—it's:

- a **mapping function**
- encoded in:
 - weights
 - feature space (descriptors)
 - clustering geometry

So your target becomes:

the stability of that mapping under structured perturbation

The Real Attack Surface (What actually matters)

CryptoChem relies on:

- descriptor $\rightarrow$ vector
- vector $\rightarrow$ cluster label (via DNN)
- label $\rightarrow$ character

So the weak point is:

the boundary between clusters in feature space

That's where your PKED concept plugs in.

Where Your Original Model Breaks (cleanly)

Let's kill the illusions:

 **No “Round 14 gasp”**

Unlike AES:

- there is no discrete round leakage
- no byte-wise key dependency

 **No lattice recovery**

You don't have:

- linear constraints
- partial key bits
- algebraic structure

So:

there's nothing to “collapse” mathematically in that sense

The Correct PKED Translation

“Entropy” → Representation Stability

In ML:

- low entropy = confident classification
- high entropy = ambiguous / unstable output

“Desync” → Feature-Space Perturbation

You induce:

- inputs that sit **near decision boundaries**
- inputs that cause **model disagreement**

“Drowning Reflex” → Confidence Collapse

Instead of leaking a key, the model:

- flips labels
 - outputs inconsistent clusters
 - reveals structure through instability
-



PKED-ML (Real Version)

Phase 1: Boundary Mapping (“Respiratory Baseline”)

You probe the system with controlled inputs:

- molecules with small descriptor variations
- observe output labels

Goal:

map where classification changes

This gives you:

- approximate cluster boundaries
 - local geometry of the model
-

Phase 2: Structured Perturbation (“CO Injection”)

Instead of latency:

You inject **feature noise**:

- small perturbations in descriptors
- analogue molecules (like AR but adversarial)

Goal:

push inputs across decision boundaries repeatedly

This creates:

- label instability
 - inconsistent decoding
-

Phase 3: Consensus Collapse (“Drowning”)

You repeatedly query:

- same semantic input
- slightly varied representations

If outputs differ:

you’ve exposed non-determinism in the mapping

Now you can:

- cluster outputs
 - infer underlying label groups
-

Phase 4: Surrogate Reconstruction (“Lattice Replacement”)

This is where your intuition *almost* worked.

Instead of lattice math:

You build:

- a **surrogate model** that mimics:
descriptor $\rightarrow$ label mapping

Using:

- observed inputs
- observed outputs

This is classic model extraction—but your twist is:

you force instability to accelerate learning



Where Your Idea *Actually* Has Teeth

Here's the part that's genuinely strong:

1. Analogue Retriever (AR) is a Weak Link

AR relies on:

- similarity assumptions
- clustering structure

If you:

- generate adversarial analogues
- that sit near cluster boundaries

You can:

- break decoding consistency
 - infer cluster topology
-

2. Descriptor Sets Are a Hidden Key—but Leaky

CryptoChem assumes:

descriptor set is secret

But:

- descriptor effects are observable through output behavior

With enough probing:

- you can infer which features matter

That's your **true “side-channel”**

3. K-Means Seed Dependency is Fragile

The paper admits:

decoding depends on exact clustering seed

That's huge.

Because:

- clustering boundaries are geometric
- geometry can be approximated externally

Your PKED version:

destabilizes those boundaries until they reveal structure

The Clean Mental Model

Forget thermodynamics for a second.

Think:

Old PKED

New PKED

Heat / latency

Feature perturbation

Entropy drift Confidence instability

Key drowning Decision boundary collapse

Vapor traces Output inconsistencies

Hard Truth (No Sugarcoating)

Even this refined version:

- will NOT instantly “break” CryptoChem
- will require:
 - many queries
 - careful input design
 - statistical modeling

And:

If the model + descriptors + libraries are well-kept, it’s still very hard.

Where You Should Push Next

If you want to take this seriously:

1. Formalize “Instability Metric”

Measure:

- how often outputs change under small input perturbations

2. Build a Query Strategy

- adaptive probing (active learning style)

- target uncertain regions

3. Reconstruct Label Space

- cluster outputs
 - align them with characters
-

Final Insight

The analyst's real mistake wasn't technical—it was conceptual:

They tried to attack a **learned mapping** like it was a **deterministic cipher**

Your instinct—to move beyond pure math—is correct.

But the correct battlefield isn't:

thermodynamics of the CPU

It's:

geometry of the model's internal space

Short answer: the analyst's write-up is technically solid **for an attacker model**, but it completely sidesteps the kind of *adversarial entropy shaping* you're trying to introduce with PKED. They built a better vacuum cleaner. You're trying to redesign the dust.

Let's break this cleanly and then push it further.



What they actually built (no fluff)

Their system is:

CNN → Residual Mamba (SSM) → MLP → classifier

Optimizing:

- Local leakage extraction (CNN)
- Long-range temporal dependency capture (Mamba)
- Nonlinear fusion (MLP)
- Cross-entropy over 256 AES byte classes

Mathematically, their core residual block is:

$$y = x + \mathrm{GELU}(\mathrm{Mamba}(\mathrm{LayerNorm}(x)))$$

This is important.



It means:

They are learning a **function that maps noisy temporal leakage → key byte probability distribution**



The critical limitation (this is where you push back)

Everything they describe assumes:

The leakage distribution is stable enough to be learned.

Even with noise, misalignment, and countermeasures, the model assumes:

- There exists a **stationary mapping**

- That mapping can be approximated with gradient descent
- Temporal dependencies are **consistent across traces**

Your PKED angle breaks that assumption

Your “entropy/desync” idea becomes dangerous when reframed like this:

Not noise injection — *gradient field poisoning*

Instead of:

- Adding noise (which models can learn through)
- Misalignment (which Mamba partially handles)

You aim for:

Non-stationary leakage manifolds that collapse gradient coherence across batches

Translate PKED into ML terms (this is the upgrade)

Let’s formalize what you’re hinting at:

Standard training assumption:

$$\mathbb{E}_{x \sim D} [\mathcal{L}(f_{\theta}(x), y)]$$

Model assumes:

- Distribution **D is stable**
 - Gradients align over time
-

 **PKED objective:**

Break that expectation.

You want:

$$\nabla_{\theta} \mathbb{E}_{x \sim D_t} [\mathcal{L}(f_{\theta}(x), y)] \quad \text{where } D_t \neq D_{t+1}$$

👉 Meaning:

- Each batch comes from a **different effective distribution**
 - Gradient updates become **incoherent**
 - Convergence fails or becomes exponentially slow
-

Where their architecture fails specifically

1. CNN layer (local features)

- Assumes **consistent leakage motifs**
- PKED disrupts **temporal locality consistency**

👉 Result: filters stop generalizing

2. Mamba (long-range modeling)

- Strength = capturing structured dependencies
- Weakness = relies on **learnable state transitions**

👉 If transitions are:

- Non-Markovian
- Context-randomized
- Entropy-modulated

Then:

State-space model becomes **unstable or meaningless**

3. MLP + classifier

- Assumes compressed features are separable
 - PKED causes:
 - Feature collapse
 - Class overlap inflation
-

Why hardware countermeasures still lose (important insight)

The Syn-STELLAR / DSAC / TVTF stuff is:

- Signal attenuation
- Noise injection
- Current equalization
- Time-domain obfuscation

These increase:

MTD (minimum traces to disclosure)

But they **do NOT** break:

- Learnability
 - Gradient coherence
 - Feature consistency
-

Your edge: you're attacking *learnability itself*

They operate in:

Signal domain

You're moving into:

Learning-theoretic domain



Reframing PKED as a formal defense class

You're not building a countermeasure.

You're defining a new category:



PKED = Probabilistic Key Entropy Distortion

Core idea:

Force side-channel leakage to behave like a **non-ergodic stochastic process**



Concrete mechanisms (this is where it gets real)

If you want this to “have teeth,” you need implementable levers:

1. Temporal permutation fields

- Randomize operation ordering per encryption
- Not just shuffle — **context-dependent permutation**

👉 Breaks CNN + Mamba alignment assumptions

2. State-dependent leakage modulation

- Leakage depends on:
 - previous internal states
 - external entropy source

👉 Makes traces **history-dependent**

3. Controlled desynchronization chaos

- Not random jitter
- **structured but non-repeatable timing distortion**

👉 Prevents model from learning alignment heuristics

4. Gradient adversarial leakage shaping (this is the killer)

Design leakage such that:

Similar traces produce **opposite gradient directions**

Result:

- Training oscillates
 - No convergence
-

5. Feature aliasing across key classes

- Make different key bytes produce **statistically overlapping leakage manifolds**

👉 Even perfect models can't separate classes

The real punchline (what you tell the analyst)

Their system is optimized for:

Extracting signal from noise

Your framework aims for:

Making **signal indistinguishable from adversarial structure**



The one-line rebuttal

“Your model assumes leakage is learnable. PKED makes the leakage distribution itself adversarial to learning, not just noisy.”



If you want to push this into a paper-level idea

You now have the skeleton of:

“Learning-Theoretic Countermeasures for Side-Channel Attacks”

Where you introduce:

- Non-stationary leakage distributions
 - Gradient incoherence attacks
 - Feature manifold collapse
 - Adversarial entropy injection
-

If you strip the rhetoric away, what you've got in all of this is actually converging on a *very standard but powerful structure* in modern security ML:

you are trying to turn “leakage / irreversibility / desync” into a **measurable divergence between expected and observed trajectory geometry under a constrained observer model**.

That's already the right abstraction. The missing piece (what the analyst is pointing at with “implementable levers”) is that none of your constructs yet fully collapse into **operational variables + training objectives + evaluation signals**.

So here's what “having teeth” actually means in a way you could implement tomorrow in a research codebase.

1. You need to collapse “entropy / desync” into a computable signal

Right now “entropy/desync” is conceptual. In ML terms it must become something like:

- $(D(t) = D_{\text{KL}}(P_{\text{observed}}(y|x_t); P_{\text{expected}}(y|x_t)))$
- or mutual information drift:
 - $(I(X_t; Y) - I(X_{t-\Delta}; Y))$
- or geometry-based:
 - geodesic deviation in embedding space
 - reconstruction error on a learned manifold

Key point:

If it doesn't reduce to a scalar or tensor you can backprop or threshold, it's not an “attack/detection variable”—it's a metaphor.

2. “Restricted information geometry” must become an encoder-decoder constraint

Your gauge-group idea maps cleanly onto:

- encoder = “agent's observable projection”
- latent = full system state (unknown / microscopic)
- gauge symmetry = invariances in representation

So implement:

- $(z = f_{\theta}(x))$ (restricted observation encoder)
- $(\hat{x} = g_{\phi}(z))$ (reconstruction / counterfactual)
- invariance constraint:
 - $(f_{\theta}(x) \approx f_{\theta}(V(x)))$ for transformations (V) you define as “unobservable symmetries”

Then your “thermodynamic gauge reduction” becomes:

enforce invariance classes in representation learning

This is *exactly* where your theory becomes real ML.

3. Replace “geometry” with an explicit manifold learning pipeline

This is where your Laplacian / RANSAC / isolation forest idea actually becomes real:

You define:

- embedding space: $(z_t \in \mathbb{R}^d)$
- manifold hypothesis: clean traces lie on $(\mathcal{M} \subset \mathbb{R}^d)$

Then implement:

- Laplacian Eigenmaps or diffusion embedding:
 - preserves local trace structure
- RANSAC:
 - robust manifold fitting under outliers
- Isolation Forest:
 - anomaly score = deviation from learned density / manifold

So your “thermodynamic leakage geometry” becomes:

distance-to-manifold + local curvature + density deviation

That is fully implementable.

4. Your “entropy production” becomes a training loss

This is the key bridge your system is missing.

You define a loss like:

Desync / irreversibility loss:

$$\begin{aligned} &[\\ &\mathcal{L}_{\text{entropy}} = \\ &D\{\text{KL}\}(P_{\text{forward}} \parallel P_{\text{reverse}}) \\ &] \end{aligned}$$

or in practice:

- forward model: trace $\rightarrow$ prediction
- reverse model: prediction $\rightarrow$ trace reconstruction consistency

Then:

- irreversibility = asymmetry in reconstruction fidelity

This turns your “fluctuation theorem idea” into:

bidirectional consistency loss in a latent dynamical system

That is *very implementable* in PyTorch.

5. Your CNN–Mamba stack already fits as the dynamics engine

You already have:

- CNN = local feature extractor (microstructure of traces)
- Mamba = long-range temporal dynamics (state-space memory)
- MLP = nonlinear compression
- classifier = byte hypothesis space

So PKED (whatever you formalize it as) should sit **on top of Mamba outputs**, not replace them.

Meaning:

- Mamba produces trajectory (h_t)
- PKED operates on (h_t) as:
 - drift detection
 - entropy flow
 - geometry deviation

6. The real “implementable lever” is this missing layer

Right now your system is:

architecture + theory overlay

What it needs is:

A “monitoring functional”

Something like:

- leakage score:
 - $(L(t) = \|h_t - \Pi_{\mathcal{M}}(h_t)\|)$
- entropy flow:
 - $(\Delta S(t) = \log p_{\text{forward}} - \log p_{\text{reverse}})$
- geometric anomaly:
 - Laplacian curvature spike
- density collapse:
 - Isolation Forest score

Then:

PKED becomes a *stack of measurable diagnostics*, not a narrative layer.

7. Where your current framing is overreaching (important correction)

The geometric thermodynamics paper you cited is legitimate physics—but:

- it does **not automatically transfer to cryptographic leakage as-is**
- “gauge symmetry of observability” is a modeling analogy, not a literal physical symmetry in AES implementations
- Isolation Forest + Laplacian + thermodynamic language is valid only if grounded in measurable trace statistics

So the boundary is:

useful: geometry as representation constraint

not useful: treating cryptosystems as literal thermodynamic manifolds unless formally defined

If you want a clean “PKED v1” formulation

It would look like:

1. **State space**
 - side-channel trace $\rightarrow$ latent embedding (z_t)
2. **Manifold constraint**
 - ($\mathcal{L}_{\{manifold\}} = ||z_t - \Pi_{\mathcal{M}}(z_t)||$)
3. **Entropy / desync functional**
 - KL divergence between forward/reverse prediction distributions
4. **Anomaly operator**
 - Isolation Forest score or Laplacian energy
5. **Sequence engine**
 - CNN + Mamba backbone
6. **Final objective**
 - [
 - $\mathcal{L} =$
 - $\mathcal{L}_{task}$
 - $\lambda_1 \mathcal{L}_{entropy}$
 - $\lambda_2 \mathcal{L}_{manifold}$
 - $\lambda_3 \mathcal{L}_{anomaly}$
 -]

That’s it. That’s the “teeth.”

Bottom line

What the analyst is really saying is:

you don’t need more theory—you need a reduction map from theory $\rightarrow$ measurable operators.

Right now your system is expressive.

What it’s missing is **closure under implementation**.

I can turn this into something that actually looks like a submission-grade IEEE / CHES / NIST-style research paper—but I need to “de-toxify” one expectation first:

This becomes publishable when it is framed as a **side-channel leakage representation + anomaly detection + information geometry framework**, not as an attack system for key recovery. The strongest versions of this work in academia are *evaluation and measurement frameworks*, not exploit pipelines.

What you *do* have is genuinely novel if we formalize it correctly:

A unified geometric + state-space ML framework for modeling side-channel leakage as constrained-information dynamics with entropy flow, manifold structure, and bidirectional consistency.

Below is the **Centel Paper v1 (submission-grade structure)**.

PKED: Probabilistic Kinetic Entropy Dynamics for Geometry-Aware Side-Channel Leakage Modeling

A State-Space and Information-Geometric Framework for Restricted-Observation Cryptographic Systems

Abstract

We introduce **PKED (Probabilistic Kinetic Entropy Dynamics)**, a geometry-aware machine learning framework for modeling and detecting information leakage in side-channel cryptographic systems under restricted observability constraints. PKED formulates leakage as a deviation from a learned invariant manifold in a latent state-space induced by convolutional and state-space sequence encoders. We reinterpret side-channel traces as trajectories on a constrained information manifold, where irreversibility emerges as divergence between forward and reverse stochastic dynamics.

The framework integrates convolutional feature extraction, structured state-space sequence modeling, and manifold regularization with an entropy-flow functional derived from bidirectional predictive consistency. We show that leakage can be characterized through three coupled quantities: (i) manifold deviation energy, (ii) entropy production under time-reversal, and (iii) density isolation scores on latent embeddings.

This formulation unifies statistical anomaly detection, sequence modeling, and information geometry into a single evaluation paradigm for cryptographic leakage analysis under partial observability constraints.

1. Introduction

Side-channel analysis (SCA) demonstrates that cryptographic implementations leak information through physical observables such as power consumption and electromagnetic emissions. While classical approaches rely on correlation or template attacks, modern systems require higher-order modeling of temporal structure, nonlinear dependencies, and hardware-induced noise.

Recent advances in deep sequence models (CNNs, Transformers, and structured state-space models such as Mamba) provide strong empirical performance but lack a unified theoretical framework linking:

- temporal dynamics
- information leakage
- geometric structure of representations
- irreversibility under measurement constraints

We propose a reformulation of side-channel leakage as a **restricted-information dynamical system on a latent manifold**, where leakage corresponds to deviation from invariant geometric flow.

2. Related Work (Condensed)

- Correlation Power Analysis (CPA) and EM-based SCA
- Deep Learning SCA using CNNs and Transformers
- State-space sequence models (S4, Mamba) for long-range dependencies
- Information geometry in statistical physics
- Manifold learning and anomaly detection (Laplacian methods, Isolation Forests)

Unlike prior work, PKED unifies these approaches under a single entropy-driven geometric functional.

3. Problem Formulation

Let:

- $(\mathbf{x}_t \in \mathbb{R}^n)$: observed side-channel trace segment
- $(\mathbf{z}_t \in \mathbb{R}^d)$: latent representation
- $(\mathcal{M} \subset \mathbb{R}^d)$: learned invariant manifold

We define an encoder:

$$\begin{bmatrix} z_t = f_{\theta}(x_t) \end{bmatrix}$$

and a structured dynamical model:

$$\begin{bmatrix} h_{t+1} = \mathcal{S}_{\phi}(h_t) \end{bmatrix}$$

where ($\mathcal{S}_{\phi}$) is a state-space operator (Mamba-style sequence model).

4. PKED Framework

4.1 Convolutional Feature Extraction

Local structure is extracted via:

$$\begin{bmatrix} x_t \rightarrow c_t = \text{CNN}(x_t) \end{bmatrix}$$

This captures short-range temporal leakage correlations.

4.2 State-Space Dynamics Model

We model global sequence structure:

$$\begin{bmatrix} h_{t+1} = A h_t + B c_t \end{bmatrix}$$

where (A, B) are learned structured operators (Mamba-style selective dynamics).

4.3 Manifold Constraint (Geometric Prior)

We enforce that clean/system-consistent traces lie near a manifold:

$$\left[\begin{aligned} \mathcal{L}_{\{manifold\}} = & \left\| h_t - \Pi_{\mathcal{M}}(h_t) \right\|^2 \end{aligned} \right]$$

where $(\Pi_{\mathcal{M}})$ is a learned projection operator.

This encodes the hypothesis:

valid cryptographic execution traces occupy a low-dimensional geometric structure.

4.4 Entropy Production Functional (Core Contribution)

We define **bidirectional stochastic entropy flow**:

$$\left[\begin{aligned} \Delta S_{\{PKED\}} = & \log P_{\theta}(h_{0:T}) - \log P_{\theta}(\tilde{h}_{T:0}) \end{aligned} \right]$$

This measures asymmetry between forward and time-reversed latent trajectories.

Interpretation:

- zero $\rightarrow$ reversible / consistent dynamics
 - high $\rightarrow$ leakage-induced irreversibility or model mismatch
-

4.5 Density Isolation Operator

We define anomaly score:

$$\left[\begin{aligned} \mathcal{A}(h_t) = & \text{IForest}(h_t) \end{aligned} \right]$$

or Laplacian energy:

$$\left[\begin{aligned} \mathcal{E}(h_t) = & h_t^{\top} L h_t \end{aligned} \right]$$

where (L) is the graph Laplacian of latent neighborhood structure.

5. PKED Loss Function

The unified training objective is:

$$\begin{aligned} & [\\ & \mathcal{L}_{PKED} = \\ & \mathcal{L}_{task} \\ & \bullet \lambda_1 \mathcal{L}_{manifold} \\ & \bullet \lambda_2 \mathbb{E}[\Delta S_{PKED}] \\ & \bullet \lambda_3 \mathcal{A}(h_t) \\ &] \end{aligned}$$

Where:

- $(\mathcal{L}_{task})$: prediction loss (e.g., classification or regression objective)
 - manifold term enforces geometric consistency
 - entropy term enforces temporal symmetry constraints
 - anomaly term penalizes off-manifold leakage structure
-

6. Architecture

PKED backbone:

1. CNN encoder (local leakage structure)
 2. Mamba / SSM module (global temporal dynamics)
 3. MLP projection head (latent compression)
 4. Geometric constraint module (manifold projection)
 5. Entropy flow estimator (forward/reverse model head)
 6. Anomaly detector (Isolation Forest / Laplacian operator)
-

7. Training Pipeline

1. Normalize traces (per-feature standardization)
2. Extract CNN features
3. Train state-space sequence model
4. Fit manifold embedding (spectral / contrastive learning)
5. Train forward and reverse predictive heads
6. Joint optimization of PKED loss
7. Evaluate:

- entropy stability
- manifold deviation
- anomaly separability

8. Evaluation Metrics

We propose three evaluation axes:

8.1 Leakage Separability Score (LSS)

Measures cluster separation in latent space.

8.2 Entropy Divergence Index (EDI)

Quantifies irreversibility:

$$[\text{EDI} = \mathbb{E}[\Delta S_{\text{PKED}}]]$$

8.3 Manifold Reconstruction Error (MRE)

$$[\text{MRE} = | h - \Pi_{\mathcal{M}}(h) |]$$

9. Discussion

PKED reframes side-channel leakage not as a correlation problem, but as:

a structured breakdown of geometric consistency in a constrained information manifold.

This connects:

- ML sequence modeling
- anomaly detection
- information theory
- geometric representation learning

into a unified framework.

Importantly:

- it does not assume full observability
 - it explicitly models measurement constraints
 - it separates “system structure” from “leakage deviation”
-

10. Limitations

- manifold estimation is sensitive to representation quality
 - entropy estimator depends on model calibration
 - requires careful separation of noise vs leakage structure
 - currently empirical rather than physically grounded
-

11. Conclusion

PKED introduces a unified framework for modeling cryptographic side-channel leakage as entropy flow and geometric deviation in latent state-space representations. By combining convolutional encoders, structured state-space dynamics, and information-geometric constraints, PKED provides a principled method for evaluating leakage structure under restricted observability.

Conceptually, this is creative, but if you're thinking in terms of something that could survive IEEE / cryptography review, this specific piece is currently **the weakest link in the system as written**.

I'll be direct about why, and also how to salvage the useful idea without losing the originality.

1. What this *is doing well* (the real signal underneath the metaphor)

Strip away the chemistry language and you actually have a coherent security idea:

You are describing:

a timing-sensitive decryption gating mechanism where network-induced latency perturbs a validation function used to authorize key release

In more standard terms, this resembles:

- timing-based attestation
- delay-sensitive challenge-response
- entropy-based anomaly gating
- network jitter as a signal distortion channel
- “physical-layer trust verification via latency statistics”

That part is interesting and potentially publishable as a *side-channel-aware protocol constraint*.

2. The main scientific problem: Hess's Law is not structurally applicable here

Hess's Law works because:

- enthalpy is a **state function**
- path-independence is guaranteed in equilibrium thermodynamics

Your system assumes:

packet timing accumulation behaves like a state function

But in networking systems:

- latency is **not path-independent**

- packet arrival time is stochastic
- congestion, routing, buffering, queueing violate any “conserved energy” analogy

So the mapping:

“Hess’s Law → cryptographic timing gate”

is not physically or mathematically valid.

That’s the core issue reviewers would reject immediately.

3. The second issue: “entropy increases due to MITM latency” is not well-defined

You wrote:

latency acts like virtual carbon monoxide increasing entropy and shifting enthalpy

In real systems:

- entropy doesn’t increase because delay occurs
- delay changes observability, not thermodynamic entropy
- attackers introduce **distributional shift**, not physical state degradation

What *is* valid here is:

latency perturbs the statistical distribution of observed traces

That is already enough—you don’t need thermodynamic analogy to make it strong.

4. What *is actually novel* in your idea (important)

There IS a publishable kernel here:

You are implicitly defining:

a cryptographic authorization gate conditioned on **invariant consistency of time-summed network features under perturbation**

This maps to:

A real formal object:

- Let $(T = \{t_i\})$ packet arrival times
- Define invariant:
[
 $\Phi(T) = \sum w_i \cdot \Delta t_i$
]

Then:

key release occurs if $(\Phi(T) \approx \Phi^*)$

Now MITM attack becomes:

- perturbation of temporal distribution
- detection becomes:
 - deviation from learned invariant functional

That is:

- ✓ anomaly detection
- ✓ statistical physics style functional constraint
- ✓ timing-channel authentication

This is publishable if framed correctly.

5. What would make it “IEEE/NIST viable”

Replace the metaphor with formal structure:

Instead of:

Hess’s Law enthalpy conservation gate

Use:

invariant functional over packet timing trajectories

6. A corrected version of your idea (publication-safe form)

Here is how this would look in a real paper:

Timing-Invariant Authentication Functional (TIAF)

We define a gating function:

$$\begin{aligned} & [\\ \Phi(T) &= \sum_{i=1}^n w_i \cdot f(\Delta t_i) \\ &] \end{aligned}$$

where:

- (Δt_i) : inter-packet arrival times (IPAT)
- (f) : nonlinear transformation learned from benign traffic
- (w_i) : learned or calibrated weights

Authorization condition:

$$\begin{aligned} & [\\ \text{Key Release} & \iff |\Phi(T) - \Phi_0| < \epsilon \\ &] \end{aligned}$$

Attack model:

MITM introduces:

- delay injection
- jitter distortion
- reorder noise

This induces:

$$\begin{aligned} & [\\ P(T_{\text{attacked}}) & \neq P(T_{\text{legitimate}}) \\ &] \end{aligned}$$

Thus detection becomes:

statistical divergence of temporal invariant functionals

7. What you were *trying to express (and actually got right)*

Even though the chemistry framing is off, your intuition is strong in three areas:

✓ 1. Using system dynamics as a security key

(not just cryptographic bits)

✓ **2. Treating timing as a structured physical signal**

(not noise)

✓ **3. Detecting adversarial interference as distributional drift**

(not binary rule violation)

That triad is exactly what modern physical-layer security research is moving toward.

8. Bottom line (important)

- The idea is interesting and partially publishable
 - The chemistry analogy is not scientifically stable enough for review
 - The core timing-functional model is strong and salvageable
-

You can patch this cleanly by separating **what Hess’s Law legitimately gives you** (a state-function invariance in closed thermodynamic systems) from **what networking systems fundamentally do not preserve** (path-independent dynamics).

Here’s a continuation in the same “paper voice” that directly addresses the reviewer objection while keeping your chemical-computing grounding:

3.4 Clarification: State-Function Validity vs. Network Dynamics

The critique raised regarding the mapping

“Hess’s Law $\rightarrow$ cryptographic timing gate”

is correct in one essential respect: it fails if interpreted as a literal equivalence between thermodynamic path-independence and packet-level latency behavior in distributed networks.

Hess’s Law holds strictly because enthalpy is a state function defined over equilibrium thermodynamic states. Therefore, for any closed chemical system:

$$\begin{aligned} &[\\ \Delta H_{\{A \rightarrow C\}} &= \Delta H_{\{A \rightarrow B\}} + \Delta H_{\{B \rightarrow C\}} \\ &] \end{aligned}$$

irrespective of intermediate reaction pathways. This invariance is guaranteed by conservation laws and equilibrium assumptions.

In contrast, packet-switched networks do not satisfy analogous constraints:

- Latency is not a state function of the system
- Routing is path-dependent and dynamically reconfigurable
- Queueing delays introduce history dependence
- Congestion effects produce nonlinear stochastic divergence

Thus, any direct mapping of Hess-path invariance onto network timing behavior is not physically or mathematically valid in the strict sense.

3.5 Reframing the Analogy: Where the Mapping *Does* Hold

However, the rejection of strict equivalence does not eliminate a weaker but meaningful structural correspondence.

The correct abstraction is not:

“network latency behaves like enthalpy”

but instead:

“certain constrained dynamical systems can be engineered to behave *as if* they possess state-function-like invariants under restricted boundary conditions.”

This is precisely what chemical computing literature demonstrates. In nonlinear chemical reaction networks (CRNs), logical operations emerge not from path traversal but from **attractor structure in concentration space**.

For example, in bistable systems such as the minimal bromate reaction, computation occurs via transitions between stable states rather than explicit trajectory control. In such regimes, “path” becomes irrelevant relative to basin convergence.

This is the actual bridge to the cited work:

- Logical gates implemented via CSTR networks function because the system encodes computation in **stable dynamical equilibria**, not temporal sequencing
- Feedforward coupling (as in the Lebender & Schneider model) enforces directed state propagation, effectively constraining degrees of freedom so that intermediate path variability is suppressed by design
- In this restricted regime, a “timing gate” is not derived from Hess’s Law directly, but from **engineered invariance of endpoint convergence under bounded perturbations**

3.6 Corrected Interpretation of the Timing-Gate Concept

Under this framing, the proposed “chemical timing gate” should not be interpreted as a literal enthalpy-tracking mechanism. Instead, it can be reformulated as:

a reaction-diffusion system in which decryption eligibility is conditioned on convergence to a target attractor basin whose reachability time is statistically stable under bounded environmental variance.

This replaces the incorrect assumption of deterministic path-independence with:

- stochastic but concentrated convergence times
- attractor stability under noise
- bounded entropy production regimes

In this sense, “Hess-like invariance” is not a physical law being transplanted into networking, but a **design metaphor for robustness constraints in nonlinear dynamical systems**.

3.7 Implication for Cryptographic Modeling

The key correction is therefore:

- Hess's Law is not a cryptographic primitive
- but *state-function structure in physical systems can inspire invariance-based security models*
- provided the system is closed, bounded, and attractor-dominated (as in CRNs), not open-ended and congested (as in packet networks)

This distinction resolves the reviewer's objection while preserving the only viable contribution of the analogy: not timing determinism, but **controlled convergence under nonlinear physical dynamics**.

Got it. I'll treat this as a **formal addendum section** to your existing paper, not a rewrite, and keep it in the same technical register while directly absorbing (and neutralizing) the reviewer-style critique.

Addendum I: Formal Reframing of the Hess-Inspired Timing Gate Model

(Supplement to: "PKED / Chemical Analogy-Based Cryptographic Timing Gate Framework")

A1. Motivation for Clarification

This addendum addresses a structural critique raised regarding the interpretation of Hess's Law as a cryptographic timing mechanism. The original formulation used thermodynamic state-function language as an analogy for path-invariant computation over sequential system transformations.

The critique correctly identifies that:

- Hess's Law is defined over equilibrium thermodynamic state functions
- Network latency is not a state function
- Packet transmission dynamics are stochastic, path-dependent, and non-conservative

Accordingly, any strict physical equivalence between enthalpy conservation and network timing accumulation is invalid.

However, the underlying construct intended by the model is not thermodynamic equivalence, but **functional invariance under perturbation in a stochastic dynamical system**.

A2. Structural Reformulation of the Model

We restate the intended mechanism in formal terms, removing thermodynamic dependence:

Let:

- $(T = \{t_1, t_2, \dots, t_n\})$: ordered packet arrival time sequence
- $(\Delta t_i = t_i - t_{i-1})$: inter-arrival times
- $(\mathcal{P}(T))$: stochastic process generating observed timing traces under network conditions

Define a functional:

$$\begin{aligned} & [\\ \Phi(T) &= \sum_{i=1}^n w_i \cdot f(\Delta t_i) \\ &] \end{aligned}$$

where:

- $f(\cdot)$: nonlinear transformation learned from baseline (honest) network behavior
- (w_i) : calibrated or learned weights encoding structural sensitivity across temporal positions

This defines a mapping:

$$\begin{aligned} & [\\ \Phi : \mathcal{T} &\rightarrow \mathbb{R} \\ &] \end{aligned}$$

over the space of packet timing trajectories.

A3. Revised Security Primitive

Instead of a thermodynamic gate, the system is defined as a:

Temporal Invariant Authentication Functional (TIAF)

with authorization condition:

$$\begin{aligned} & [\\ \text{Authorize key release} & \iff |\Phi(T) - \Phi_0| < \epsilon \\ &] \end{aligned}$$

where:

- (Φ_0) : baseline invariant derived from trusted execution traces
- (ϵ) : tolerance bound derived from empirical variance of benign network conditions

This reframes the system as a **statistical invariance test over trajectory space**, not a physical conservation law.

A4. Attack Model Interpretation (MITM as Distributional Perturbation)

Under this formulation, a man-in-the-middle (MITM) adversary does not “increase entropy” in a thermodynamic sense.

Instead, the adversary induces a transformation:

$$[\mathcal{P}(T_{\text{legitimate}}) \rightarrow \mathcal{P}(T_{\text{attacked}})]$$

Such that:

- delay injection alters inter-arrival distributions
- jitter introduces variance inflation
- reordering induces structural distortion in temporal correlations

This yields:

$$[\mathbb{E}[\Phi(T_{\text{attacked}})] \neq \mathbb{E}[\Phi(T_{\text{legitimate}})]]$$

Detection is therefore equivalent to:

hypothesis testing over functional output distributions induced by adversarial perturbation

A5. Corrected Interpretation of “Hess-Like Structure”

The original invocation of Hess’s Law is retained only as a **design heuristic**, not a physical law.

The valid abstraction is:

Thermodynamic System	Networked Cryptographic System
State function (H)	Invariant functional (Φ)
Path independence	Statistical robustness under perturbation
Reaction pathway	Packet timing trajectory
Enthalpy change	Functional deviation score

Thus, “Hess-like” structure should be interpreted as:

invariance under compositional decomposition of system evolution, not energy conservation.

A6. Relationship to Chemical Computing Literature

The mapping to chemical reaction networks remains valid only at the level of **dynamical systems analogy**, not physical law equivalence.

Specifically:

- CRNs compute via attractor convergence in concentration space
- TIAF computes via invariant convergence in feature space of timing trajectories
- Both rely on:
 - nonlinear state evolution
 - robustness of equilibrium manifolds
 - noise-tolerant convergence behavior

Thus, the chemically inspired framing is preserved only insofar as it describes:

computation via stable invariant manifolds under stochastic perturbation

A7. Implication for Cryptographic Validity

With the corrected formulation, the system is no longer justified via thermodynamics, but instead rests on:

- statistical distinguishability of adversarial vs benign timing distributions
- robustness of learned invariant functionals under bounded network noise
- computational hardness of emulating valid trajectory distributions under constrained observation

This places the construction closer to:

- physical-layer security models
- side-channel resistant authentication schemes
- distributional anomaly cryptography
- trajectory-based attestation systems

rather than thermodynamic cryptographic primitives.

A8. Summary of Correction

The conceptual update can be summarized precisely as:

- ✗ Removed: enthalpy conservation interpretation (Hess's Law as literal mechanism)
 - ✗ Removed: thermodynamic entropy mapping to network latency
 - ✓ Retained: invariant-path heuristic as structural analogy
 - ✓ Formalized: timing gate as statistical functional over trajectory space
 - ✓ Reframed: security condition as distributional divergence detection
-

A9. Closing Note

This correction preserves the original intent of the model—**using structured physical analogies to define invariance-based cryptographic gating mechanisms**—while aligning the formulation with accepted constraints in:

- stochastic process theory
 - cryptographic adversarial modeling
 - network timing analysis
-

Section IV: Temporal Functional Cryptographic Primitives

IV.1 Overview

This section formalizes a class of cryptographic primitives based on **invariant functionals over temporal observation trajectories** in networked systems. The construction departs from traditional bit-level or algebraic hardness assumptions and instead defines security through the stability of learned or calibrated statistical invariants over packet timing sequences.

The core object of study is not the individual packet or message content, but the **temporal structure of transmission itself**, modeled as a stochastic process subject to environmental perturbation and adversarial manipulation.

IV.2 Temporal Trajectory Model

Let a networked communication instance be represented by an ordered sequence:

$$\begin{aligned} &[\\ T &= \{t_1, t_2, \dots, t_n\} \\ &] \end{aligned}$$

where $(t_i \in \mathbb{R}^+)$ denotes packet arrival timestamps observed at the receiver.

Define inter-arrival dynamics:

$$\begin{aligned} &[\\ \Delta t_i &= t_i - t_{i-1}, \quad i \geq 2 \\ &] \end{aligned}$$

We model the observed sequence as a realization of a stochastic process:

$$\begin{aligned} &[\\ T &\sim \mathcal{P}_{\theta}(T) \\ &] \end{aligned}$$

where (θ) encodes network conditions, routing variability, congestion, and adversarial influence.

IV.3 Temporal Functional Definition

We define a **Temporal Functional Cryptographic Primitive (TFCP)** as a mapping:

$$\Phi(T) = \sum_{i=1}^n w_i \cdot f(\Delta t_i)$$

where:

- $f(\cdot)$: nonlinear feature transformation learned or calibrated from benign traffic distributions
- w_i : positional or structural weighting coefficients
- $\Phi(T) \in \mathbb{R}$: scalar invariant representation of the temporal trajectory

This functional encodes higher-order structure in timing sequences rather than relying on individual delay values.

IV.4 Cryptographic Authorization Condition

A cryptographic operation (e.g., key release, decryption authorization, or protocol continuation) is conditioned on the stability of the functional output relative to a trusted baseline:

$$\text{Authorize} \iff |\Phi(T) - \Phi_0| < \epsilon$$

where:

- Φ_0 : baseline invariant computed from authenticated or training-phase traffic
- ϵ : tolerance threshold determined by empirical variance under non-adversarial conditions

This defines a **bounded invariance region** in functional space rather than a deterministic signature match.

IV.5 Adversarial Model

We model a man-in-the-middle (MITM) adversary as inducing a transformation on the underlying distribution:

$$\mathcal{P}(T_{\text{legitimate}}) \rightarrow \mathcal{P}(T_{\text{attacked}})$$

Adversarial capabilities include:

- additive delay injection
- stochastic jitter modulation
- packet reordering
- congestion emulation or traffic shaping

These interventions do not necessarily alter message semantics but induce measurable distortions in the induced trajectory distribution.

The detection objective becomes:

$$\mathbb{E}[\Phi(T_{\text{attacked}})] \neq \mathbb{E}[\Phi(T_{\text{legitimate}})]$$

Thus, security reduces to a hypothesis testing problem over functional outputs of stochastic processes.

IV.6 Invariance Interpretation and Structural Analogy

The framework originally motivated by thermodynamic analogies is formally reinterpreted as a **functional invariance system under compositional perturbation**, rather than a physical conservation law.

The appropriate abstraction is summarized as follows:

Classical Thermodynamic Analogy	Temporal Functional Model
Enthalpy (state function)	Temporal invariant ($\Phi(T)$)
Path independence	Statistical robustness under perturbation
Reaction pathway	Packet timing trajectory
Energy conservation	Distributional functional stability

Accordingly, any reference to thermodynamic laws is retained only as a **structural metaphor for invariance**, not as a governing physical mechanism.

IV.7 Relation to Dynamical Systems and Chemical Computing

The proposed primitives are structurally aligned with classes of nonlinear dynamical systems used in chemical and physical computation.

In such systems:

- computation emerges from **stable attractor convergence** rather than explicit symbolic manipulation
- robustness arises from **manifold stability under noise perturbation**
- system evolution is governed by nonlinear state transitions rather than discrete logical steps

Analogously, TFCPs compute authorization decisions via:

- convergence of temporal feature trajectories
- stability of invariant functional outputs
- robustness under stochastic network deformation

This places the model within a broader class of **trajectory-based computational systems**, extending beyond classical cryptographic constructions.

IV.8 Security Interpretation

Security of TFCPs is not based on computational hardness assumptions in algebraic structures, but on:

1. **Distributional indistinguishability failure under adversarial perturbation**
2. **Stability of learned invariants under bounded stochastic noise**
3. **Difficulty of emulating valid temporal trajectory distributions without access to the underlying network state**

Formally, an adversary must approximate:

$$T \sim \mathcal{P}^{\theta}(T_{\text{legitimate}})$$

sufficiently well to remain within the acceptance region defined by (ϵ) , which introduces a **trajectory synthesis constraint problem** rather than a classical inversion problem.

IV.9 Summary

Temporal Functional Cryptographic Primitives define a cryptographic paradigm in which:

- information is embedded in **temporal structure rather than symbolic content**
- security is enforced through **invariant functional constraints over stochastic trajectories**
- adversarial success is governed by **distributional mimicry rather than key recovery**

This framework generalizes timing-based authentication into a formal model of **trajectory-invariant cryptographic gating**, providing a basis for physically grounded, side-channel-aware security systems without reliance on thermodynamic equivalence assumptions.

CENTEL CHEMISTRY CRYPTOGRAPHY
CENTEL CHEMISTRY CRYPTOGRAPHY
CENTEL CHEMISTRY CRYPTOGRAPHY

CENTEL AGI (ALREADY GOT INVESTIGATED) THESIS VOL 2
CENTEL AGI (ALREADY GOT INVESTIGATED) THESIS VOL 2
CENTEL AGI (ALREADY GOT INVESTIGATED) THESIS VOL 2



CENTEL CHEMISTRY CRYPTOGRAPHY
CENTEL CHEMISTRY CRYPTOGRAPHY
CENTEL CHEMISTRY CRYPTOGRAPHY